Zurich University of Applied Sciences
Department Life Sciences and Facility Management
Institute of Computational Life Sciences

BACHELOR THESIS

Inferring mutations from single cell transcriptomics of kidney's filtering system

Author:
Riccardo Viscio

Supervisors:
Dr. Tugçe Bilgin Sonay
Prof. Dr. Maria Anisimova

Submitted on
July 16, 2026

Study program:
Applied Digital Life Sciences, B.Sc.

Imprint

Project: Bachelor Thesis
Title: Inferring mutations from single cell transcriptomics of kidney's filtering system
Author: Riccardo Viscio
Date: July 16, 2026
Keywords: Choose Keywordss
Copyright: Zurich University of Applied Sciences

Study program:
Applied Digital Life Sciences, B.Sc.
Zurich University of Applied Sciences

Supervisor 1:
Dr. Tugçe Bilgin Sonay
Zurich University of Applied Sciences
Email: bilg@zhaw.ch
Web: Link

Supervisor 2:
Prof. Dr. Maria Anisimova
Zurich University of Applied Sciences
Email: maria.anisimova@zhaw.ch
Web: Link

Abstract

Write an Abstract

Zusammenfassung

Zusammenfassung in Deutsch

Acknowledgements

I extend my sincere thanks to Dr. Tugçe Bilgin Sonay, for her invaluable guidance, expertise, and commitment throughout this project. I am also grateful to Olesia Kondrateva for her significant intellectual contributions and ideas, which were vital to this project's development.

Contents

Abstract

Zusammenfassung

Acknowledgements

List of Abbreviations

1 Introduction and Theoretical Background **1**
1.1 Somatic SNVs: Definition and Relevance 1
1.2 Bulk and Single-Cell RNA Sequencing Approaches 2
1.3 Smart-seq2 as a Platform for Variant Calling 3
1.4 Challenges of SNV Detection from scRNA-seq 3
1.5 Variant Calling Tools 4
1.6 Research Question and Objectives 4

2 Methodology **5**
2.1 Dataset and Biological Context 5
2.2 Pipeline Overview 5
2.3 Alignment 6
2.4 Variant Calling and Quality Filtering 6
2.5 Variant Calling Matrix Construction (Not sure if necessary) 6
2.6 Further Analysis (tbd) 6

3 Results **7**
3.1 Pipeline Performance 7
3.2 Tool comparison 7
3.3 SNV Landscape 7
3.4 Further Results 7

4 Discussion **8**
4.1 Limitations 8
4.2 Tool Suitability for Smart-seq2 8

5 Conclusion **9**
5.1 Summmary 9
5.2 Outlook 9

Bibliography **10**

Overview of Applied AI Tools **12**

Code Availability and Repository Structure **13**

List of Figures

List of Tables

A Project Directory Structure

List of Abbreviations

AI	Artificial Intelligence
UMI	Unique Molecular Identification
scRNA-seq	Single-Cell RNA Sequencing
NGS	Next-Generation Sequencing
NGS	Next-Generation Sequencing
PCR	Polymerase Chain Reaction
SNV	Single Nucleotide Variant
SV	Structural Variant
SNP	Single Nucleotide Polymorphism
VAF	Variant Allele Fraction
indels	insertions and deletions
PCA	Principal Component Analysis
POLE	DNA Polymerase Epsilon
SLURM	Simple Linux Utility for Resource Management
TSV	Tab-Separated Values

Chapter 1

Introduction and Theoretical Background

1.1 Somatic SNVs: Definition and Relevance

Genetic variation is the molecular foundation of biological diversity, adaptation, and disease. At the broadest level, two fundamental classes of genetic variation shape phenotypic diversity and disease susceptibility in any organism: those inherited through the germline, and those acquired somatically during an individual's lifetime.

Germline variants are present from conception, encoded in every cell of the organism, and passed from parent to offspring through reproduction. Their population frequencies are shaped over generations by evolutionary forces such as natural selection, genetic drift, and recombination, and they underlie a broad spectrum of heritable conditions, from monogenic disorders to complex polygenic disease risk [1].

Somatic mutations, by contrast, do not follow Mendelian inheritance. They arise during an individual's lifetime in individual somatic cells as a consequence of DNA damage or errors in DNA replication, and are propagated exclusively to the clonal descendants of the founding cell [1]. Because they originate in a single cell and spread only through cell division, somatic mutations are present in only a subset of an organism's cells.

Genetic variants can be divided into four principal classes: **single-nucleotide variants** (SNVs), which are point substitutions at individual genomic positions; short insertions and deletions (**indels**, $<$50 bp); **structural variants** (SVs), encompassing large deletions, duplications, inversions, and translocations; and large **chromosomal abnormalities**, such as whole-chromosome gains or losses [1, 2]. Among these, SNVs are the most abundant and best characterised class. The most common type of SNV, the single-nucleotide polymorphism (SNP), refers to positions at which heritable variation occurs at appreciable frequency in a population; the term SNV is broader, encompassing both germline and somatic single-base substitutions [3].

This distinction has a direct technical consequence: because a somatic mutation is present in only a fraction of cells, it appears in sequencing data at a low **variant allele fraction** (VAF). This low VAF is both the defining hallmark of somatic mutations and the core technical challenge in their detection, as sequecing noise and artefacts can

occur at comparable frequencies, making confident discrimination between signal and error non-trivial [**MuyasDenovodetection2024**, 2].

Somatic SNVs arise from two broad categories of mechanism: errors introduced during DNA replication, and DNA damage that is incorrectly or incompletely repaired. During each cell division, the replicative DNA polymerases (*Pol* δ and *Pol* ε) incorporate a small number of incorrect nucleotides that escape proofreading correction and become permanently fixed in the daughter cell's genome [4]. These replication-coupled errors produce a characteristic baseline mutation rate that accumulates in proportion to the number of cell divisions a lineage has undergone.

Understanding the principles governing somatic SNV generation, accumulation, and detection therefore forms an essential prerequisite for the computational approaches employed in this thesis, where somatic mutations are inferred from Smart-seq2 single-cell RNA sequencing data of mouse kidney without a matched DNA reference.

1.2 Bulk and Single-Cell RNA Sequencing Approaches

Bulk RNA-seq is an **next-generation sequencing** (NGS)-based technology used to measure the transcriptome of a cell population. Rather than resolving individual cells, it takes a whole-population approach by pooling many cells together into a single sample, yielding a combined gene expression profile. The resulting measurements therefore reflect the average expression level of each gene across all cells in the sample. While powerful for detecting differential expression between conditions, this averaging fundamentally obscures cell-to-cell variation. Rare cell populations, transient states, and cell-type-specific signals are diluted or invisible in the aggregate readout [5, 6].

Single-cell RNA sequencing (scRNA-seq) resolves this limitation by capturing the transcriptome of individual cells, enabling the identification of distinct cell types, developmental trajectories, and heterogeneous expression states within a tissue. Beyond gene expression, the single-cell resolution of scRNA-seq data has opened the possibility of inferring genetic variation, including somatic SNVs, directly from transcriptomic reads, repurposing expression data for genomic analysis without additional sequencing experiments [5, 7].

Two principal scRNA-seq paradigms have emerged that differ fundamentally in their technical design. Droplet-based methods, such as the widely used 10X Genomics Chromium platform, encapsulate individual cells in microfluidic droplets alongside barcoded beads, enabling highly multiplexed profiling of tens of thousands of cells per experiment. However, they capture only the 3′ end of transcripts and rely on unique molecular identifiers (UMIs) for quantification. Plate-based methods, exemplified by Smart-seq2, instead isolate individual cells into wells and generate full-length cDNA libraries across the entire transcript body [8, 9]. This full-length coverage comes at the cost of lower throughput, but provides uniform read depth across transcripts.

1.3 Smart-seq2 as a Platform for Variant Calling

The distinction between 3′-end and full-length transcript coverage is not merely a matter of quantification accuracy but it has fundamental implication for genomic variant detection. Droplet-based methods such as 10X Genomics concentrate sequencing reads near the 3′ terminus of transcripts, leaving the vast majority of each transcript's sequence unobserved. This severely restricts the genomic positions at which variants can be detected, as only SNVs residing close to the 3′ end of expressed genes will be covered by any reads [9]. Smart-seq2, by generating full-length cDNA libraries, distributes reads across the entire transcript body, offering access to variants at any expressed position and yielding higher per-gene read depth for low-abundance transcripts [8, 9].

Despite these advantages, Smart-seq2 remains comparatively underexplored as a variant calling platform. The field has rapidly adopted droplet-based methods such as 10X, driven by their scalability and substantially lower per-cell cost [9], and the majority of computational tools for scRNA-seq variant calling have been benchmarked and developed primarily on data from such platforms [7]. Furthermore, Smart-seq2 lacks UMIs, meaning that **Polymerase Chain Reaction** (PCR) duplicates cannot be collapsed by molecular barcode, a property that complicates the separation of true variants from amplification artefacts [8]. Nevertheless, the uniform, full-length coverage of Smart-seq2 provides a richer substrate for variant calling than 3′-biased methods, making it a valuable platform for inferring somatic SNVs from transcriptomic data.

1.4 Challenges of SNV Detection from scRNA-seq

Calling SNVs from scRNA-seq data is fundamentally more challenging than from bulk or single-cell DNA sequencing. Because RNA sequencing captures only the expressed fraction of the genome, variant detection is limited to transcribed regions and further restricted by variable expression levels across cells, lowly expressed genes yield insufficient read coverage for confident variant calling, and genes not expressed in a given cell produce no reads at all [7, 10]. This sparse and uneven coverage contrasts sharply with bulk DNA sequencing, where depth can be uniformly controlled across genomic regions of interest.

Beyond coverage, scRNA-seq reads carry additional sources of noise absent in DNA sequencing. Errors introduced during reverse transcription, misalignments at splice junctions, and RNA editing events can all produce apparent base changes that mimic genuine SNVs [**MuyasDenovodetection2024**, 11]. At the single-cell level, the low number of RNA molecules per cell further amplifies stochastic noise, making it difficult to distinguish true low-frequency variants from technical artefacts without matched DNA data for reference. In the absence of a paired germline sample, somatic and germline variants cannot be separated by direct comparison, requiring instead the use of population-level databases and statistical models applied across the cell population [**MuyasDenovodetection2024**, 2]. These constraints collectively motivate the need for dedicated computational tools designed specifically for the statistical properties of scRNA-seq data.

1.5 Variant Calling Tools

We've decdided to use or bowtie as well as STAR. the reason for that is that none of the proposed tools fitted our needd this also reflects the fact that the scope of the project is still not that well established and quite new

1.6 Research Question and Objectives

- What is the research question? s look at the disposition.

- What is the aim of the project?

Look at the disposition

Chapter 2

Methodology

2.1 Dataset and Biological Context

The dataset used was from ADISAS paper, in the study she studied the KLOTHO a paper. We've received the data because it was already available. The data is from the nephron of the mice. It was primary used to detect the KLOTHO protein. we've used the data due to comodity.

The dataset was is based of 373 different cell all coming from one mouse.

the source of the dataset is from the KLOTHO study. in this study it has been discussed whether Klotho protein is moderately expressed in kidney proximal tubule and moreon (DC), which includes the distal convoluted abundant in the distal convolutitubule (DCT) and connecting tubule (CNT). However, the function of Klotho in the DC, particularly its role in sKlotho release and regulation of mineral metabolism, remains unclear.

2.2 Pipeline Overview

Due to the scarsity of suitable tools or pipelines found before. We've decided. The following aligner ... for the pipeline.

The pipeline started with raw unmapped reads these files were provied as fastq files. we then have mapped the raw files to the reference genome GRCm38/mm10). Additionally for the STAR aligner the GTAK files were needed these were taken from here `https://ftp.ebi.ac.uk/pub/databases/gencode/Gencode_mouse/release_M25/gencode.vM25.annotation.gtf.gz` these were the annotation. The annotation is quite important for STAR because it needs it due to the splice-aware nature of the aligner: STAR uses the GTF annotation to build a database of known splice junctions during index generation, enabling it to correctly map reads that span exon–exon boundaries rather than discarding them as unalignable or soft-clipping them at junctions.

first we downloaded the dataset from the webpage due to high data storage this was done on the HPC with the help of a slurm script.

the data was processed with the help of a nextflow pipeline there we've decdided to use two aligners one time we've used bowtie2 secondly we've used the star alinger in this pipeline we've followin steps were executed

- Reads were trimmed using `fastx_trimmer` (FASTX-Toolkit), retaining bases at positions 20 to 126 (`-f 20`, `-l 126`), removing low-quality ends from both the 5′ and 3′ ends of each read.

- Quality filtering was applied with `fastq_quality_filter`, retaining only reads where at least 95% of bases (`-p 95`) had a Phred quality score $Q \geq 30$ (`-q 30`). The Phred quality score is defined as:

$$Q = -10 \cdot \log_{10}(P_e) \tag{2.1}$$

  where P_e is the probability of an incorrect base call. A threshold of $Q = 30$ corresponds to $P_e = 10^{-3}$, i.e. a base call accuracy of 99.9%.

- Reads were aligned to the mm10 reference genome using `STAR` (`-runMode alignReads`), with a splice junction overhang of 125 bp (`-sjdbOverhang 125`) and a suffix array index size of 14 (`-genomeSAindexNbases 14`). The genome index was pre-built (`-runMode genomeGenerate`) using the mm10 FASTA and GTF annotation. Output was produced as coordinate-sorted BAM (`-outSAMtype BAM SortedByCoordinate`) using 8 threads (`-runThreadN 8`).

- Variants were called using `bcftools mpileup` to compute per-position genotype likelihoods from the aligned BAM, followed by `bcftools call` (`-mv`) to retain only variant sites, outputting results in uncompressed VCF format (`-Ov`).

INSERT MERMAID SCRIPT

2.3 Alignment

2.4 Variant Calling and Quality Filtering

2.5 Variant Calling Matrix Construction (Not sure if necessary)

2.6 Further Analysis (tbd)

Chapter 3

Results

Include the whole research Question discussed on the 07.04 primers and so on

3.1 Pipeline Performance

3.2 Tool comparison

3.3 SNV Landscape

3.4 Further Results

Chapter 4

Discussion

4.1 Limitations

A big error was that I first mapped the data to the wrong genome. I did not take into reference that the given pipeline was mapping to the human mitochondria. At first the pipeline worked also quite fast. however, we quickly found out that this was wrong. I then mapped the data to the right data.Mapping the data to the right genome was quit an insight, which showed me that i should have questioned more and not taken so much as granted

4.2 Tool Suitability for Smart-seq2

Chapter 5

Conclusion

5.1 Summmary

5.2 Outlook

Bibliography

[1] Zhi Yu et al. "Genetic Variation across and within Individuals". In: *Nature Reviews Genetics* 25.8 (Aug. 2024), pp. 548–562. ISSN: 1471-0064. DOI: 10.1038/s41576-024-00709-x. URL: https://www.nature.com/articles/s41576-024-00709-x (visited on 04/12/2026).

[2] Chang Xu. "A Review of Somatic Single Nucleotide Variant Calling Algorithms for Next-Generation Sequencing Data". In: *Computational and Structural Biotechnology Journal* 16 (Feb. 6, 2018), pp. 15–24. ISSN: 2001-0370. DOI: 10.1016/j.csbj.2018.01.003. PMID: 29552334. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC5852328/ (visited on 04/12/2026).

[3] Hecun Zou et al. "Significance of Single-Nucleotide Variants in Long Intergenic Non-protein Coding RNAs". In: *Frontiers in Cell and Developmental Biology* 8 (May 25, 2020), p. 347. ISSN: 2296-634X. DOI: 10.3389/fcell.2020.00347. PMID: 32523949. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC7261909/ (visited on 04/12/2026).

[4] Vivian S. Park and Zachary F. Pursell. "POLE Proofreading Defects: Contributions to Mutagenesis and Cancer". In: *DNA Repair* 76 (Apr. 2019), pp. 50–59. ISSN: 15687864. DOI: 10.1016/j.dnarep.2019.02.007. URL: https://linkinghub.elsevier.com/retrieve/pii/S1568786418303185 (visited on 10/12/2025).

[5] Yunjin Li et al. "Exploring Additional Valuable Information From Single-Cell RNA-Seq Data". In: *Frontiers in Cell and Developmental Biology* 8 (Dec. 1, 2020), p. 593007. ISSN: 2296-634X. DOI: 10.3389/fcell.2020.593007. PMID: 33335900. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC7736616/ (visited on 03/01/2026).

[6] Xinmin Li and Cun-Yu Wang. "From Bulk, Single-Cell to Spatial RNA Sequencing". In: *International Journal of Oral Science* 13.1 (Nov. 15, 2021), p. 36. ISSN: 2049-3169. DOI: 10.1038/s41368-021-00146-0. URL: https://www.nature.com/articles/s41368-021-00146-0 (visited on 04/12/2026).

[7] Fenglin Liu et al. "Systematic Comparative Analysis of Single-Nucleotide Variant Detection Methods from Single-Cell RNA Sequencing Data". In: *Genome Biology* 20.1 (Nov. 19, 2019), p. 242. ISSN: 1474-760X. DOI: 10.1186/s13059-019-1863-4. URL: https://doi.org/10.1186/s13059-019-1863-4 (visited on 02/26/2026).

[8] Simone Picelli et al. "Full-Length RNA-seq from Single Cells Using Smart-seq2". In: *Nature protocols* 9 (Jan. 2, 2014), pp. 171–181. DOI: 10.1038/nprot.2014.006.

[9] Xiliang Wang et al. "Direct Comparative Analyses of 10X Genomics Chromium and Smart-seq2". In: *Genomics, Proteomics & Bioinformatics*. Single-Cell Omics Analysis 19.2 (Apr. 1, 2021), pp. 253–266. ISSN: 1672-0229. DOI: 10.1016/

j.gpb.2020.02.005. URL: https://www.sciencedirect.com/science/article/pii/S1672022921000486 (visited on 04/12/2026).

[10] Trung Nghia Vu et al. "Cell-Level Somatic Mutation Detection from Single-Cell RNA Sequencing". In: *Bioinformatics* 35.22 (Nov. 15, 2019), pp. 4679–4687. ISSN: 1367-4803. DOI: 10.1093/bioinformatics/btz288. PMID: 31028395. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC6853710/ (visited on 02/24/2026).

[11] Valérie Marot-Lassauzaie et al. "Identifying Cancer Cells from Calling Single-Nucleotide Variants in scRNA-seq Data". In: *Bioinformatics* 40.9 (Sept. 1, 2024), btae512. ISSN: 1367-4811. DOI: 10.1093/bioinformatics/btae512. URL: https://doi.org/10.1093/bioinformatics/btae512 (visited on 02/22/2026).

Overview of Applied AI Tools

- **Anthropic (2025).** *Claude Pro (Sonnet 4.5).*
  Available at: https://claude.ai/
  Applications: Coding assistance and programming support, including code structure optimization, debugging, algorithm explanation, and enhancing understanding of best practices. Also used for text-related tasks including generating, editing, and refining academic, professional, and creative content.

- **DeepL (2025).** *DeepL Translator and DeepL Write.*
  Available at: https://www.deepl.com/translator
  Applications: Text translation, grammar correction, synonym search, translation between German and English, and suggestions for academic writing.

- **Grammarly (2025).** *Grammarly Premium.*
  Available at: https://www.grammarly.com/
  Applications: Grammar correction, sentence restructuring, and suggestions to improve clarity and tone.

- **Microsoft (2025).** *Microsoft Copilot.*
  Available at: https://www.microsoft.com/en-us/microsoft-copilot
  Applications: Coding assistance and debugging support for Python and general programming tasks. Assisted with the clarification of scientific and medical topics and provided LaTeX formatting support.

- **NotebookLM (2025).** *NotebookLM.*
  Available at: https://notebooklm.google.com/
  Applications: Searching and summarizing relevant information contained within uploaded documents (e.g., research papers, lecture notes). Used to extract key ideas, clarify theoretical concepts, and create structured summaries. Supports understanding of domain-specific content by providing contextual explanations based on user-supplied sources.

- **Perplexity (2025).** *Perplexity AI.*
  Available at: https://www.perplexity.ai/
  Applications: Assisting with research by retrieving and summarizing information from diverse sources, including academic and scientific materials. Used to explore unfamiliar topics, gather context for literature understanding, and obtain concise explanations of technical concepts. Supports guided question–answer workflows for efficient knowledge acquisition.

Code Availability and Repository Structure

All code used for data preprocessing, model inference, machine learning, and statistical analysis is available in the following public repository:

Code Repository: github.com/acg-team/tumor-segmentation-ucec-stad

The repository is organized to facilitate reproducibility, with distinct directories for data handling, model inference, and downstream analysis.

Repository Structure

The project structure is outlined below, highlighting the core components required to reproduce the study:

```
tumor-segmentation-ucec-stad/
|-- data/
|   |-- raw/                  # Image lists and clinical metadata
|   +-- processed_csv/        # Aggregated cell-level data
|-- hover_net/                # Original HoVer-Net model implementation
|-- hover_next_inference/     # Optimized HoVer-Next inference pipeline
|-- notebooks/
|   |-- exploratory/          # Descriptive statistics
|   |-- machine_learning/     # Machine Learning pipeline
|   |-- statistical_analysis/ # Statistical tests
|   +-- cluster/              # Clustering algorithms and results
|-- scripts/
|   |-- inference/            # HPC batch scripts for GPU inference
|   |-- machine_learning/     # Scripts for PCA, training, evaluation
|   |-- preprocessing/        # CSV splitting and merging utilities
|   |-- processing/           # Heatmap generation tools
|   +-- visualization/        # Tile visualization scripts
+-- src/
    |-- dataset_processing/   # Tile-level data aggregation
    +-- image_processing/     # Heatmap generation and IO
```

Key Components

Model Inference

Two deep learning models were utilized for nuclei segmentation and classification:

- **HoVer-Net:** The original architecture used for baseline segmentation.
- **HoVer-Next:** A lightweight, optimized variant used to accelerate processing on large Whole Slide Images (WSIs). The inference logic and environment

setup are located in the `hover_next_inference/` directory.

Data Processing Pipeline

The `src/` and `scripts/` directories contain the complete workflow for handling WSI data:

- **Preprocessing:** Scripts to filter images by molecular subtype (e.g., `filter_tmb_h.py`) and manage large CSV datasets.
- **Heatmap Generation:** Tools to visualize cellular density and distribution directly on WSI thumbnails (e.g., `process_heatmap.sh`).
- **Data Aggregation:** The `combine_tiles_csv.py` script aggregates millions of individual cell detections into case-level summaries for statistical analysis.

Machine Learning & Analysis

The downstream analysis is centralized in the `notebooks/` directory:

- **Exploratory Analysis:** Notebooks for descriptive statistics and initial data validation.
- **Machine Learning Pipeline:** End-to-end scripts for dimensionality reduction (`pca_reduction.py`), model training (`train_models.py`), and performance evaluation (`evaluate.py`).
- **Statistical Testing:** Jupyter notebooks containing Mann-Whitney U tests, Kruskal-Wallis tests, and correlation matrix generation.

Environment and Dependencies

To ensure reproducibility, all software dependencies are specified in the root environment file:

- `hovernet_env.yml`: Defines the Conda environment, including PyTorch, Scikit-learn, and image processing libraries required to run the pipeline.

List of Figures

List of Tables

Appendix A

Project Directory Structure

```
.
|-- data
|   |-- processed_csv
|   |   |-- case_level_aggregated.csv
|   |   |-- combined_wsi_data.csv
|   |   |-- msi-h.csv
|   |   |-- msi-h.zip
|   |   |-- mss_tmb-h.csv
|   |   |-- mss_tmb-l.csv
|   |   |-- mss_tmb-l_part1.zip
|   |   |-- mss_tmb-l_part2.zip
|   |   |-- processed_csv.zip
|   |   +-- tmb-h.zip
|   |-- raw
|   |   |-- msi-h_image_list.csv
|   |   |-- mss_tmb-l_image_list.csv
|   |   +-- tmb_h_image_list.csv
|   |-- README.md
|   |-- stad_clinical.csv
|   +-- ucec_clinical.csv
|-- hover_net
|-- hover_next_inference
|-- ec_env.yml
|-- notebooks
|   |-- cluster
|   |   |-- clustering.py
|   |   |-- clustering_results
|   |   +-- logs
|   |-- exploratory
|   |   |-- exploratory_analysis.ipynb
|   |   +-- images
|   |-- machine_learning
|   |   |-- configs
|   |   |-- data
|   |   |-- logs
|   |   |-- ml_results
|   |   |-- README.md
|   |   +-- scripts
|   |-- README.MD
|   +-- statistical_analysis
|       |-- images
|       |-- statistical_analysis.ipynb
|       +-- statistical_tests
```

```
|-- README.md
|-- results
|   |-- hovernet
|   |   |-- msi-h
|   |   +-- tmb_h
|   +-- hovernext
|       |-- msi-h
|       +-- mss_tmb-l
|-- scripts
|   |-- clustering
|   |   +-- run_clustering.sh
|   |-- create_links
|   |   |-- submit_filtered_img.sh
|   |   |-- submit_hovernet.sh
|   |   +-- submit_linked_img.sh
|   |-- inference
|   |   |-- hovernet_a100.sh
|   |   |-- hovernet_l40s.sh
|   |   |-- hovernext_a100.sh
|   |   |-- hovernext_l40s.sh
|   |   +-- README.md
|   |-- machine_learning
|   |   |-- evaluation.sh
|   |   |-- learning_curves.sh
|   |   |-- README.md
|   |   |-- run_pca.sh
|   |   +-- train.sh
|   |-- preprocessing
|   |   |-- combine_csv.sh
|   |   |-- merge_csv.sh
|   |   +-- split_csv.sh
|   |-- processing
|   |   |-- process_heatmap_csv.sh
|   |   |-- process_heatmap.sh
|   |   +-- README.md
|   |-- README.MD
|   |-- utils
|   |   |-- check_unprocessed_hovernet.sh
|   |   |-- check_unprocessed_hovernext.sh
|   |   +-- zip_folders.sh
|   +-- visualization
|       |-- visualize_hovernet_tiles.sh
|       +-- visualize_hovernext_tiles.sh
+-- src
    |-- dataset_processing
    |   |-- combine_tiles_csv.py
    |   |-- create_image_list.py
    |   |-- csv_operations.py
    |   |-- __init__.py
    |   +-- README.md
    |-- image_processing
    |   |-- constants.py
    |   |-- export.py
    |   |-- heatmap.py
    |   |-- __init__.py
    |   |-- io_utils.py
    |   |-- main.py
```

```
    |   |-- __pycache__
    |   |-- README.md
    |   |-- tile_analysis.py
    |   |-- visualize_multiple_tiles.py
    |   |-- visualize_tile_hovernet.py
    |   +-- visualize_tile_hovernext.py
    |-- __init__.py
    +-- README.MD

40 directories, 67 files
```